



Guidelines for programming assignment #1

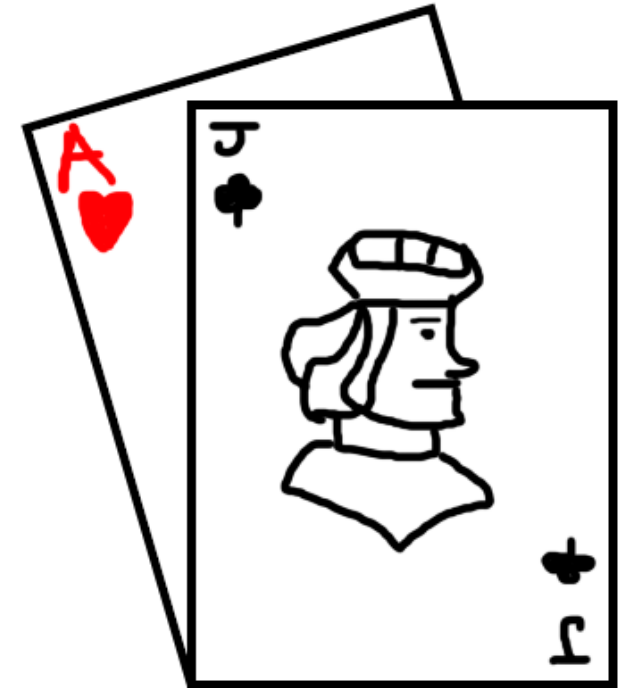
INFO0010: Introduction to computer networking
Academic year 2026-2027

Practical details

- Email: davan.chiemdao@uliege.be
- Office: B28 1.105
- Appointments at my office must have been *scheduled* beforehand
- Ecampus' forum will be the communication channel
 - Question must be asked on the platform
 - Check other threads before posting

Context

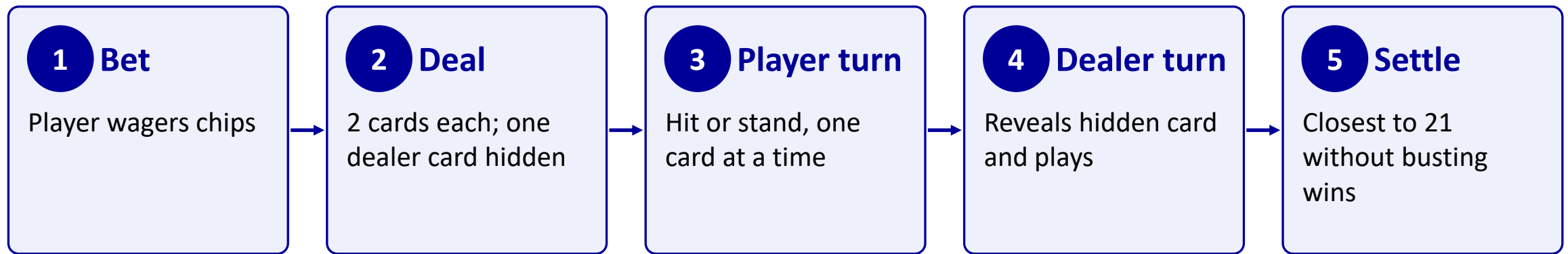
- You will develop a client/server application: a **Blackjack (21)** platform
- The server is a game platform; each client plays against the house
- Java 17 Sockets (TCP)
- Console output (no fancy GUI)
- Imposed protocol: the *Blackjack Protocol* (BP)
- To be realized alone
- **HARD deadline:** October 25th, 2026



Blackjack = 21

Blackjack rules: a round

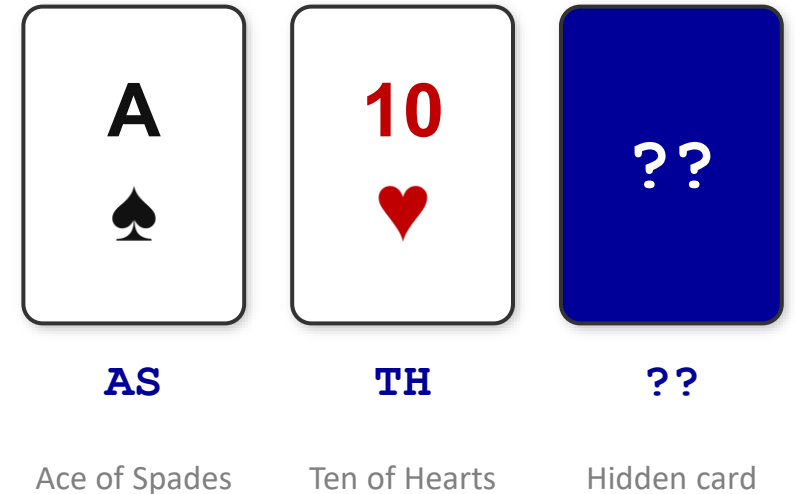
- On connection: the server shuffles *one 52-card deck* and sets the bankroll to *100 chips*
- The deck is never reshuffled during the session



- The session ends when the user *quits*, *runs out of chips*, or the deck holds *fewer than 4 cards*
- Between rounds the user places a new bet to start the next round

Cards and hand values

- A card is a **rank** followed by a **suit** letter, exactly 2 characters
 - Rank: 2–9, T (Ten), J, Q, K, A
 - Suit: S (Spades), H (Hearts), D (Diamonds), C (Clubs)
- A hidden (face-down) card is written ??
- Values: 2–9 face value, T/J/Q/K = 10
 - Ace = 11, unless the hand would exceed 21: then 1
 - A total is always the best achievable value
- *Natural Blackjack*: Ace + ten-value card as the initial two cards



Player actions and dealer play

Action	Available
Bet	between rounds; starts a new round
Hit	only during a round: draw a card
Stand	only during a round: end your turn
Bank	at any time: check the bankroll
Quit	at any time: end the session

- No splitting of pairs, no doubling down
- Player total > 21 after a hit: **bust**, the round is lost immediately

Dealer: fixed strategy

≤ 16

dealer hits

≥ 17

dealer stands

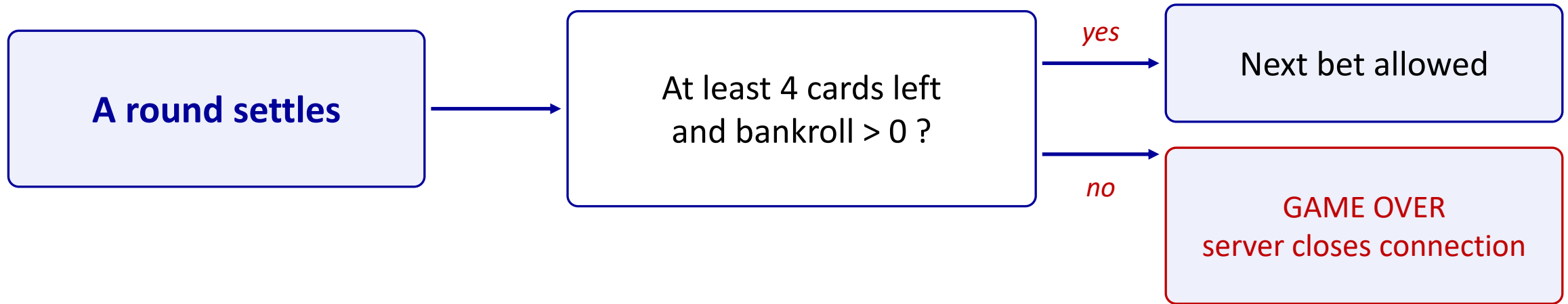
- Dealer busts: the player wins
- Player busts: the dealer draws nothing, its hidden card is just revealed

Payouts

Outcome	When	Chips
<i>BLACKJACK</i>	Natural 21 on the deal, without any hit	3:2 , rounded down (bet 5 wins 7)
<i>WIN</i>	Higher total than the dealer without busting, or dealer busts	1:1
<i>PUSH</i>	Same total as the dealer (also both naturals)	bet returned
<i>LOSE</i>	Player busts, dealer higher, or dealer natural and player none	bet forfeited

- The bet is deducted from the bankroll when the round starts and settled when it is resolved

End of the game



- The response that settles the round also reports the end of the session
- No reshuffle ever occurs; a bet cannot be placed with a bankroll of 0
- The deck can run out *during* a round (last card taken by a hit or dealer draw):
 - Player hit without busting: the round is settled as if a stand had followed
 - Dealer needs a card that does not exist: it stops drawing, the round settles as it stands

Protocol

- Client connects to the server on the port given (e.g. localhost on port 2123)
- The **Blackjack Protocol** (BP) is custom-made and *text-oriented*: human-readable, uppercase keywords
- Multiline protocol: every line ends in «CRLF» (`\r\n`)
- A whole request or response ends with an *empty line*: it finishes with `\r\n\r\n`
- A single CRLF ends a line inside a message; the blank line marks the message boundary

Example

```
C: BET 10\r\n\r\nS: PLAYER AS KH 21\r\nDEALER 9C 7D 16\r\nBLACKJACK 15\r\n\r\n
```

C: = client request, S: = server response

Client requests

Request	Meaning	Server answers
BET amount	wager and deal a new round; amount = strictly positive base-10 integer, no leading zeros	Hand (+ RoundStatus, GameOver)
HIT	draw an additional card	Hand (+ RoundStatus, GameOver)
STAND	end the player's turn, dealer plays	Hand + RoundStatus (+ GameOver)
BANK	query the bankroll	BANKROLL amount
QUIT	end the session	closes the connection
anything else		WRONG, connection stays open

Every request ends with CRLF

Server responses

- **Hand:** the two lines
 - `PLAYER card1 card2 ... total`
 - `DEALER card1 card2 ... total`
 - Hidden card: `??`, and the total is `??` too
- **RoundStatus**, only when the round is resolved
 - `BLACKJACK amount, WIN amount, LOSE amount, PUSH 0`
- **GameOver**, after RoundStatus, then the connection is closed
 - `GAME OVER` followed by `BANKROLL amount`

Round in progress

```
PLAYER 8D 6S 14
DEALER 5H ??
```

Round resolved

```
PLAYER 9S 6C 15
DEALER JD 8H 18
LOSE 125
GAME OVER
BANKROLL 0
```

Each message ends with a blank line

Errors and priorities

Request	Condition	Response
BET amount	amount is not a NUMBER	INVALID BET
BET amount	amount exceeds the bankroll	INSUFFICIENT FUNDS
BET amount	a round is already in progress	ROUND IN PROGRESS
HIT / STAND	no round in progress	NO ACTIVE ROUND
anything else	unknown request	WRONG (connection kept open)

- The three BET checks are in *decreasing priority*: an invalid or too large bet is reported even if a round is in progress
- A successful BET deducts the bet, deals the cards and answers with Hand

Example session

```
C: BET 10
S: PLAYER AS KH 21
   DEALER 9C 7D 16
   BLACKJACK 15
```

```
C: BANK
S: BANKROLL 115
```

```
C: BET 10
S: PLAYER 8D 6S 14
   DEALER 5H ??
```

```
C: HIT
S: PLAYER 8D 6S 4C 18
   DEALER 5H ??
```

```
C: STAND
S: PLAYER 8D 6S 4C 18
   DEALER 5H 9D QC 24
   WIN 10
```

```
C: BET 125
S: PLAYER 9S 6C 15
   DEALER JD ??
```

```
C: STAND
S: PLAYER 9S 6C 15
   DEALER JD 8H 18
   LOSE 125
   GAME OVER
   BANKROLL 0
```

Blue lines are client requests; every message ends with CRLF

(Un)intentional malevolence

- What happens if I send «BET abc», «HIT» before any bet, or «FOLD»?
 - Good behaviour: answer `INVALID BET, NO ACTIVE ROUND` or `WRONG`
 - Bad behaviour: trigger an exception, or crash
 - *Never expect, always check!*
- What happens if I send «BET 10» but never the blank line?
 - The server waits for the rest of the request, that never comes
 - If single-threaded, it cannot handle new connections
 - One thread for new connections, then one thread per connection
 - Use socket time-outs (`setSoTimeout()`, triggers an exception), close the connection if too long

Stream-oriented protocols (TCP)

- TCP gives a *reliable, in-order byte stream*: nothing lost, duplicated or reordered
- Output may be *buffered*: call `flush()` on the `OutputStream`
- One `write` is not necessarily one `read` on the other side

Sent as two requests, received as three chunks:

```
BET 10\r\n\r\nHIT\r\n\r\n
```



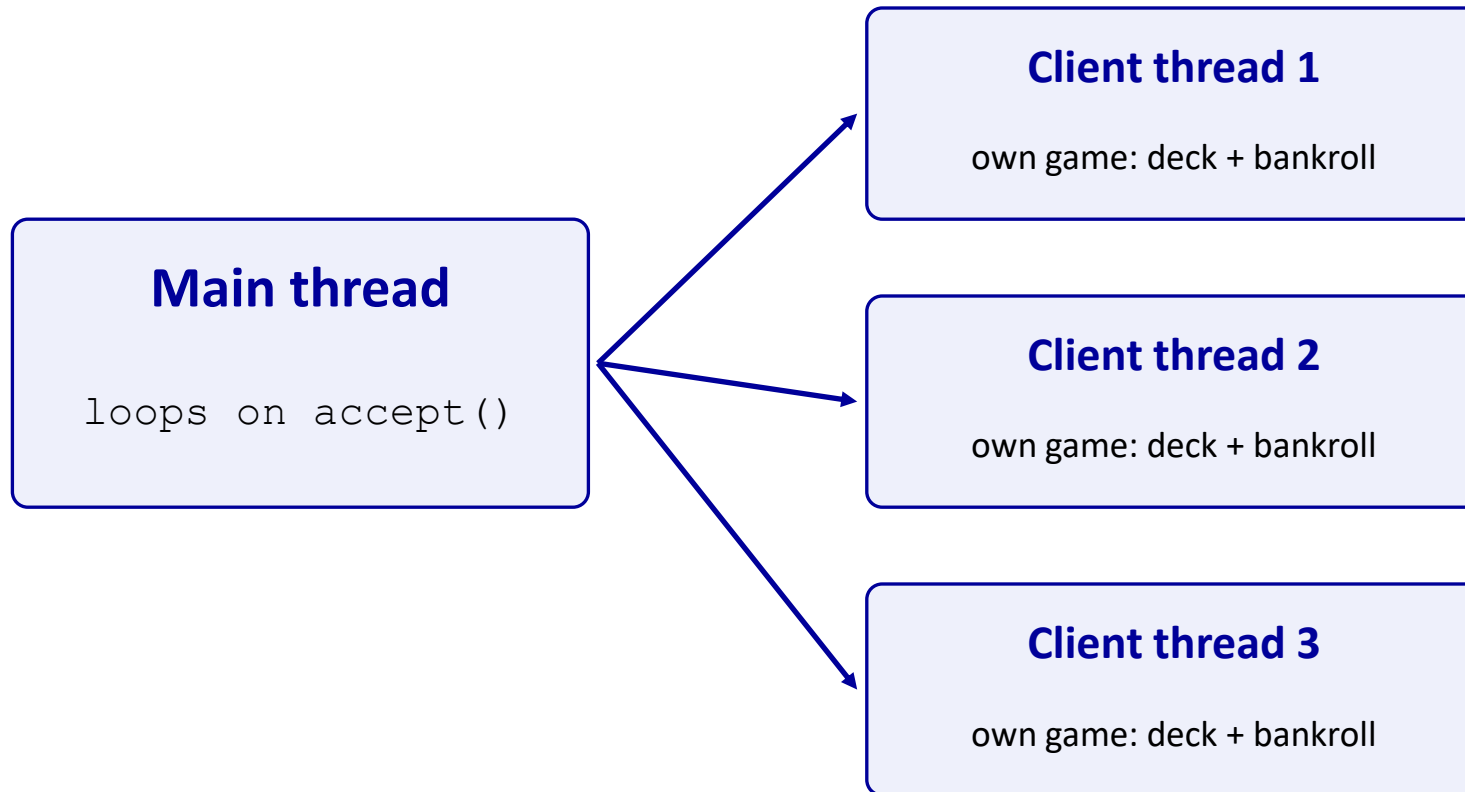
BET 1

0\r\n\r\nHI

T\r\n\r\n

BP marks message boundaries with a blank line: use a class that handles multiple reads (e.g. `BufferedReader`)

Multi-threading



- One session = one TCP connection = one game
- A slow or erroneous client only blocks its own thread
- `setSoTimeout()` on the socket: a client silent for too long triggers an exception
- Do threads share state? If yes, *synchronize* them
- This design has a flaw, seen in part 2; acceptable for part 1

Testing and report

Testing: show evidence

- Tools: debugger, Wireshark/tcpdump, raw netcat/telnet, automated tests, custom client
- Per scenario: what you did and what you observed
- Explain how to reproduce: commands, inputs, expected output
- Cover rules, edge cases, malicious behaviour, protocol

Report: PDF, 4 pages max

- Follows the provided template
- *Software architecture*: major classes and roles
- *TCP stream*: how you read the requests
- *Multi-thread organisation* and synchronization
- *Robustness*: exceptions, misbehaving clients
- *Tests*: tools and how to reproduce

Extra guidelines

- Java 17, packages `java.lang`, `java.io`, `java.net`, `java.util` only
- Port number: `2xxx`, where `xxx` = last 3 digits of your ULiege ID
- Main classes `BlackjackServer` and `BlackjackClient`, in `BlackjackServer.java` / `BlackjackClient.java` at the root of the archive
 - Launched with no argument: `java BlackjackServer`, `java BlackjackClient`
- No `package` instruction, no sub-directories, no shutdown hooks, no file manipulation
- Program must stop with CTRL+C at any time
- Fully operational: you are encouraged to test on student machines
- Submit code + report as a .zip on the Montefiore Submission Platform (submit.montefiore.uliege.be) before **October 25th, 2026**
- Oral exam at the end of the semester: **know your own code**. Insufficient understanding may result in point reduction.

Plagiarism/Fraud

- Definition of plagiarism:

"an act or instance of using or closely imitating the language and thoughts of another author without authorization and the representation of that author's work as one's own, as by not crediting the original author"

source: <http://dictionary.reference.com/browse/plagiarism>

- Copying (even parts of) another student's source code, or an external source without proper quoting, is *plagiarism*
- Putting (even parts of) your source code at the disposal of other students is a *fraud*
- Both are sanctioned with a grade of **0/20** for the course (and possibly other sanctions)
- A copied and cited external part is allowed, but awards no points
- In case of suspicion, an oral examination will determine if the project is the student's work

Plagiarism vs. cooperation



Fraud

Copy without quoting

Sharing its source code



Danger zone

Discussing the content of the
source code

Quote the source code of another
student's project



Cooperation

Quote an external source

Discussing the behaviour of the
program

Charter for the use of AI tools by students

- AI tools can be used for this project if you clearly disclose how you used them and can fully explain and defend everything you submit.
- Using CEUR-WS Policy on AI-Assisting Tools Students, student are required to declare their use of AI for any part of the project: research, implementation and/or report
- The [Declaration on Generative AI](#), as part of the report, states either:
 - “The author(s) have not employed any Generative AI tools” or
 - Using the [activity taxonomy](#): “During the preparation of this work, the author(s) used XXX to: ..., ..., and Further, the author(s) used YYY for figures 3 and 4 to: After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication’s content.”